Ingegneria del Software — Soluzione del Tema 2/09/2021

Esercizio 1

Si consideri la seguente classe Java `PhotoLibrary` per la gestione di una collezione di foto. Ogni foto è rappresentata dalla classe immutabile `Photo` che contiene alcune informazioni tra cui l'insieme delle persone (classe immutabile `Person`) che appaiono nella foto.

```
public class PhotoLibrary {
  // Ritorna il numero di foto contenute nella collezione.
  public /*@ pure @*/ int size();

  // Ritorna l'i-esima foto contenuta nella collezione (in ordine di inserimento).
  public /*@ pure @*/ Photo get(int i);

  // Aggiunge una foto alla collezione.
  // Lancia una DuplicateException se la foto e' gia' presente nella collezione.
  public void add(Photo photo) throws DuplicateException;

  // Ritorna le foto che contengono tutte le persone in s.
  // s deve contenere almeno una persona.
  public /*@ pure @ */ Set<Photo> getPhotosWith(Set<Person> s);

  // Ritorna la lista di tutte le persone contenute in foto della collezione. La lista e'
  // ordinata dalla persona che appare in piu' foto alla persona che appare in meno foto
  // (in caso di uguale numero di apparizioni, l'ordine non e' specificato).
  public /*@ pure @ */ List<Person> getAllPeople();
}

public /*@ pure @*/ class Photo {
  // Ritorna l'insieme di persone contenute nella foto (puo' essere vuoto).
  public Set<Person> getPeople();
  ... }
```

Domanda a)

Si specifichino in JML i metodi `add`, `getPhotosWith`, `getAllPeople`.

Soluzione

```
//@requires photo != null
//@
//@ensures (\forall int i; i>=0 && i<\old(size()); !\old(get(i)).equals(photo)) &&
//@ size() == \old(size())+1 && get(size()-1).equals(photo) &&
//@ (\forall int i; i>=0 && i<\old(size()); \old(get(i)).equals(get(i)))
//@
//@signals (DuplicateException e)
//@ (\exists int i; i>=0 && i<\old(size()); \old(get(i)).equals(photo)) &&
//@ size() == \old(size()) &&
//@ (\forall int i; i>=0 && i<\old(size()); \old(get(i)).equals(get(i)))
public void add(Photo photo) throws DuplicateException;

//@requires s != null && !s.isEmpty()
//@
//@ensures \result != null && (\forall Photo p; ; \result.contains(p) <==>
//@ (\exists int i; i>=0 && i<size(); get(i).equals(p) && p.getPeople().containsAll(s)) )
public /*@ pure @ */ Set<Photo> getPhotosWith(Set<Person> s);

//@ensures \result != null &&
//@ (\forall Person p; ; \result.contains(p) <==>
//@   (\exists int i; i>=0 && i<size(); get(i).getPeople().contains(p)) ) &&
//@ (\forall int i; i>=0 && i<\result.size();
//@   (\forall int j; j>=0 && j<i; !\result.get(i).equals(\result.get(j))) )
//@ (\forall int i; i>=0 && i<\result.size()-1;
//@   (\numof int j; j>=0 && j<size(); get(j).getPeople().contains(\result.get(i))) >=
//@   (\numof int j; j>=0 && j<size(); get(j).getPeople().contains(\result.get(i+1)))
public /*@ pure @ */ List<Person> getAllPeople();
```

Domanda b)

Si consideri un'implementazione che utilizza una `List` per contenere le foto della collezione, come mostrato di seguito.

```
public class PhotoLibrary {
  private final List<Photo> photos;
  ...
}
```

Per tale implementazione si definiscano l'invariante di rappresentazione e la funzione di astrazione.

Soluzione

Nell'invariante di rappresentazione escludiamo che la `List` sia nulla, che contenga valori nulli e che contenga duplicati.

```
//@(* Representation invariant RI *)
//@private invariant photos != null && !photos.contains(null) &&
//@ (\forall int i; i>=0 && i<photos.size();
//@   (\forall int j; j>=0 && j<i; !photos.get(i).equals(photos.get(j))) )
```

La funzione di astrazione definisce `size()` e `get()` in funzione della `List` usata nella rappresentazione. Gli altri metodi pubblici sono infatti definiti a partire da essi.

```
//@(* Abstraction function AF *)
//@private invariant size() == photos.size() &&
//@ (\forall int i; i>=0 && i<photos.size(); photos.get(i).equals(get(i)))
```

Domanda c)

Si consideri una classe `PhotoLibraryIter` che aggiunge un metodo `iter` che ritorna un iteratore `Iterator<Photo>` per iterare sulle foto della collezione. `PhotoLibraryIter` può essere definita come sottoclasse di `PhotoLibrary` nel caso in cui l'iteratore ritornato non definisca il metodo (opzionale) `remove`? E nel caso in cui definisca tale metodo?

Soluzione

Il metodo `iter` è puro, ma se il motodo `remove` è definito, l'iteratore può modificare la collezioni rimuovendo foto. Tale rimozione violerebbe una proprietà invariante della classe `PhotoLibrary`, che assume che le foto non possano essere rimosse. Di conseguenza `PhotoLibraryIter` può essere definita come sottoclasse di `PhotoLibrary` solo nel caso in cui il metodo `remove` non sia definito.

Esercizio 2

Si consideri la seguente classe Java:

```
public class WeatherSensor {
  private double temp;
  private double humidity;
  public synchronized double getTemp() { return temp; }
  public synchronized double getHumidity() { return humidity; }
  public synchronized void setTemp(double temp) { this.temp = temp; }
  public synchronized void setHumidity(double humidity) { this.humidity = humidity; }
}
```

Domanda a)

Si consideri il seguente frammento di codice che crea due thread che accedono (uno in scrittura ed uno in lettura) alla medesima istanza condivisa della classe `WeatherSensor`:

```
WeatherSensor ws = new WeatherSensor();

new Thread() {
  public void run() {
    for(int i=0; i<10000; i++) {
      ws.setTemp(i); ws.setHumidity(i);
    }
  }
}.start();

new Thread() {
  public void run() {
    double t = ws.getTemp();
    double h = ws.getHumidity();
    if(t == h) System.out.println("Same");
    else System.out.println("Different");
  }
}.start();
```

E' corretto affermare che il secondo thread stamperà sempre `Same`? In caso non fosse così si modifichi il frammento di codice (senza toccare la classe `WeatherSensor`) affinchè sia certo che venga sempre stampato `Same`.

Soluzione Il frammento di codice va modificato come segue:

```
WeatherSensor ws = new WeatherSensor();

new Thread() {
  public void run() {
    for(int i=0; i<10000; i++) {
      synchronized(ws) {
        ws.setTemp(i); ws.setHumidity(i);
      }
    }
  }
}.start();

new Thread() {
  public void run() {
    synchronized(ws) {
      double t = ws.getTemp();
      double h = ws.getHumidity();
    }
    if(t == h) System.out.println("Same");
    else System.out.println("Different");
  }
}.start();
```

Domanda b)

Si consideri il seguente metodo, parte della classe `WeatherTemp`, che ritorna `true` se la temperatura del sensore corrente e del sensore `other` sono uguali e `false` altrimenti:

```
public synchronized boolean sameTemp(WeatherSensor other) {
  return this.temp == other.temp;
}
```

Il metodo è correttamente sincronizzato? Spiegare perchè e nel caso non lo fosse fornire una versione correttamente sincronizzata del metodo.

Soluzione

Il metodo non acquisisce il lock su `other` pur accedendo all'attributo privato `other.temp`. Va corretto come segue:

```
public boolean sameTempOk(WeatherSensor other) {
  double t;
  synchronized(other) { t = other.temp; }
  synchronized(this) { return this.temp == t; }
}
```

Domanda c)

Si aggiunga alla classe `WeatherSensor` un ulteriore metodo `getPositiveTemp` che restituisce la temperatura del sensore solo se positiva, sospendendo il chiamante fino a quando la temperatura non diventa positiva. Si indichi se e quali altri metodi della classe `WeatherSensor` vadano modificati e come.

Soluzione

```
public synchronized double getPositiveTemp() throws InterruptedException {
  while(temp <= 0) wait();
  return temp;
}
```

inoltre va aggiunta l'istruzione `notifyAll()` al metodo `setTemp`.

Esercizio 3

Si consideri il seguente programma Java.

```
class Veicolo {
   protected void accensione(int tempoMax) {
      System.out.println("accensione-Veicolo in "+tempoMax);
   }
   public void spegnimento() {
      System.out.println("spengo-Veicolo");
   }
}

class VeicoloAMotore extends Veicolo {
   public void accensione(int tempoMax) {
      System.out.println("accensione-VeicoloAMotore in "+tempoMax);
   }
   public void spegnimento(int tempo) {
      System.out.println("spengo-VeicoloAMotore");
   }
}

public class Motociclo extends VeicoloAMotore {
   public void accensione(double tempoMax) {
      System.out.println("accensione-Motociclo in "+tempoMax);
   }
   public void spegnimento() {
      System.out.println("spengo-MotoCiclo");
   }
   public static void main(String[] args)  { ...}
}
```

Il main è costituito dalle seguenti istruzioni, numerate per riferimento:

```
1  Veicolo v = new Veicolo();
2  Veicolo v1= new VeicoloAMotore();
3  VeicoloAMotore vm1= new Veicolo();
4  VeicoloAMotore vm2= new Motociclo();
5  v.accensione(3);
6  v1.accensione(3);
7  vm1.accensione(3);
8  vm2.accensione(3);
9  v.spegnimento(1);
10 v.spegnimento();
11 v1.spegnimento();
12 v1.spegnimento(2);
13 vm1.spegnimento(2);
14 vm2.spegnimento();
15 vm2.spegnimento(2);
```

Domanda a)

Scrivere il numero di riga delle istruzioni (se ne esistono) che generano un errore in compilazione. Giustificare brevemente la risposta.

Soluzione

Righe 3, 7, 9, 12,13.

La riga 3, in quanto Veicolo non è una sottoclasse di VeicoloAMotore. Quindi devono essere eliminate anche le righe 7 e 13. Le righe 9 e 12, in quanto un Veicolo non possiede il metodo spegnimento(int).

Domanda b)

Supponendo di eliminare le eventuali righe che generano errori, si scriva, per ogni riga numerata, il valore stampato in uscita.

Soluzione

5: accensione-Veicolo in 3
6: accensione-VeicoloAMotore in 3
8: accensione-VeicoloAMotore in 3
10: spengo-Veicolo
11: spengo-Veicolo
14: spengo-MotoCiclo
15: spengo-VeicoloAMotore

NB: La riga 8 chiama il metodo VeicoloAMotore.accensione(int) in quanto il valore 3 è un int, non un double.

Esercizio 4

Si considerino i seguenti metodi Java e se ne completino le parti omesse, *utilizzando escusivamente i concetti e i costrutti della programmazione funzionale*.

Domanda a)

Il metodo seguente prende come argomento una lista di nomi (di tipo `String`) (il parametro `people`) e restituisce il nome di maggiore lunghezza fra quelli interamente in maiuscolo. Per esempio, se la lista include i nomi "Paolo", "MARIO", "ISA", "Antonio", il metodo restituisce "MARIO". Se non esiste alcun nome insteramente in maiuscolo, il metodo restituisce la stringa "n.a.".

Si ricorda che la classe `String` ha il metodo `length()` che restituisce la lunghezza della stringa.

```
public static  String longestInUppercase(List<String> people) {
   return ....;
}
```

Suggerimento: per verificare se un nome è interamente in maiuscolo si può ad esempio usare il metodo `toUpperCase()` della classe `String`, che restituisce la conversione in maiuscolo della stringa this – un semplice confronto del risultato con l'originale può stabilire se l'originale era già interamente in maiuscolo.

Soluzione

```
public static  String longestInUppercase(List<String> people) {
   return people.stream()
                .filter(name -> name.equals(name.toUpperCase()))
                .reduce((name1,name2) -> name1.length()>=name2.length() ?name1 : name2)
                .orElse("n.a.");
}
```

Domanda b)

Il metodo seguente prende ancora come argomento una lista di nomi di tipo `String` (il parametro `people`), ma restituisce la lista dei nomi che non sono interamente in maiuscolo, convertiti in maiuscolo. Per esempio, se la lista include i nomi "Paolo", "MARIO", "ISA", "Antonio", il metodo restituisce una lista composta da "PAOLO", "ANTONIO".

```
public static  List<String> inUpperCase(List<String> people) {
   return ....;
}
```

Soluzione

```
public static  List<String> inUpperCase(List<String> people) {
  return people.stream()
               .filter(name -> !name.equals(name.toUpperCase()))
               .map(name -> name.toUpperCase())
               .collect(Collectors.toList());
}
```